

```
1 // 情况 B: &arr
2 □ int(*p2)[5] = &arr; // 注意这里 p2 的类型变了！
3 // p2 指向整个数组。
4 // p2 + 1 走 20 个字节（整个数组的大小）。
```

这个具体是什么意思啊详细讲讲呗，然后告诉我这个的用法

深入理解数组指针：int (*p2)[5]

这是一个非常深入且高级的问题！这正是 C 语言指针最烧脑、也最精髓的地方：指针的步长。

我们要理解 int (*p2)[5]，核心在于理解两件事：

- 1. 它长什么样？（语法层面）
- 2. 它怎么走？（内存层面）

一、核心概念：指针的“步长”由谁决定？

指针变量里存的是一个地址（比如 0x1000）。

当我们对指针进行 +1 运算时，指针并不是真的只加 1 个字节，而是加上它指向的“目标”的大小。

- 如果指向 int: +1 就跳过 4 个字节。
- 如果指向 double: +1 就跳过 8 个字节。
- 如果指向一个包含 5 个元素的数组？ +1 就跳过整个数组（5 * 4 = 20 字节）。

二、代码深度解析

让我们拆解这行代码：

```
1 int arr[5] = { 1, 2, 3, 4, 5 };
2 int(*p2)[5] = &arr;
```

1. 右边: &arr 是什么？

- arr 是数组名。
- &arr 取的是整个数组的地址。
- 虽然数值上，&arr 和 &arr[0] 是一样的（都是数组首字节地址），但意义完全不同。
 - &arr[0]: 指向“第一块砖”。
 - &arr: 指向“整栋房子”。

2. 左边: int (*p2)[5] 是什么？

这是一个数组指针。我们需要仔细拆分它的声明语法（这是一个考点）：

- (*p2): p2 首先是一个指针变量。
- [5]: 它指向的目标是一个包含 5 个元素的数组。
- int: 这个数组里的元素是 int 类型。

注意小括号的重要性：

- int (*p2)[5]：数组指针（p2 是指针，指向数组）。
- int *p2[5]：指针数组（p2 是数组，存了 5 个指针）。
一定要看清楚小括号括住的是谁！

3. 它是怎么走的？（图解）

假设 arr 的起始地址是 0x1000。

```
1 printf("p2 的地址: %p\n", p2); // 输出 0x1000
2 printf("p2 + 1 的地址: %p\n", p2 + 1); // 输出多少？
```

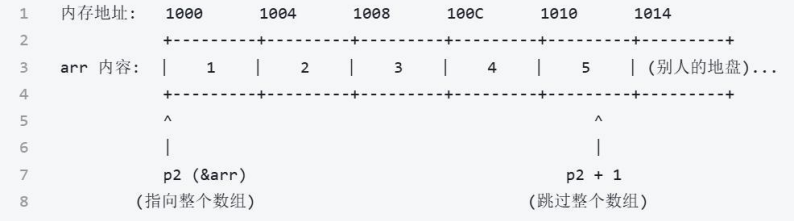
计算过程：

p2 指向的是一个“长度为 5 的 int 数组”。

大小 = 5 * sizeof(int) = 5 * 4 = 20 字节。

所以 p2 + 1 实际上是 0x1000 + 20 = 0x1014。

图解如下：



结论：p2 把那 5 个元素看成是一个整体“包裹”，+1 就是直接跳过这个包裹，去往下一个包裹。

三、这个东西有什么用？（实战用法）

你可能会问：“我为什么要跳过整个数组？我平时都是一个个元素访问的啊。”

确实，在处理一维数组时，int *p = arr 更好用。数组指针 int (*p)[5] 主要用于处理二维数组。

用法：处理二维数组

假设有一个二维数组：

```
1 int matrix[3][5] = {
2     {1, 2, 3, 4, 5}, // 第 0 行
3     {6, 7, 8, 9, 10}, // 第 1 行
4     {11, 12, 13, 14, 15} // 第 2 行
5 };
```

在内存中，二维数组是按行连续存储的。

我们可以把 matrix 看作是“由 3 个‘大包裹’组成的序列”，每个“大包裹”就是一个含有 5 个元素的一维数组。

这时候，数组指针就派上用场了：

```

1  #include <stdio.h>
2
3  int main() {
4      int matrix[3][5] = { 0 }; // 定义二维数组
5
6      // 【用法】
7      // p 是一个指针，指向一个“含有 5 个 int 的数组”
8      int(*p)[5] = matrix;
9
10     // 解释：
11     // matrix 是二维数组名，它 decay（退化）为指向第 0 行的指针。
12     // 第 0 行就是一个含有 5 个 int 的数组。
13     // 所以类型刚好匹配！
14
15     // 遍历二维数组
16     for (int i = 0; i < 3; i++) {
17         // p + i 指向第 i 行（跳过了 i 个“大包裹”）
18         // *(p + i) 拿到了第 i 行的数组名（退化成首元素地址）
19         // 所以 *(p + i) 相当于 matrix[i]
20
21         printf("第 %d 行首元素: %d\n", i, *(p + i)[0]);
22         // 或者更常见的写法：
23         // printf("%d ", p[i][0]);
24     }
25
26     return 0;
27 }
28

```

对比一下：

如果你试图用普通的 `int *p` 来指向二维数组，代码会报错或很难处理：

```

1  int* p_err = matrix; // 警告！类型不兼容
2  // 因为 matrix 指向的是“一行”，而 p_err 指向的是“一个整数”。
3

```

四、总结

1. `int (*p)[5]` 叫做**数组指针**。
2. 它把整个数组看作一个“原子单位”，+1 就跳过整个数组（跳过 5 个元素）。
3. **主要用途**：用于处理二维数组或者多维数组，用来指向其中的“一行”。

记忆口诀

括号包指针，指针指数组；

步长跨整行，遍历二维数。

在 sizeof 和 & 运算符中使用数组名 arr 时，分别会出现什么情况

深入理解数组名：sizeof 与 & 的特殊例外

这是一个非常经典且重要的 C 语言面试题/考点。

在 C 语言中，数组名 arr 通常情况下会“退化”为指向首元素的指针。但在两个例外情况下，数组名**不退化**，保持它“数组”的本色：

- 4. sizeof(arr)
- 5. &arr

我们来详细拆解这两种情况。

情况一：sizeof(arr)

结果： 得到整个数组的字节大小。

- **不退化：** 当 sizeof 后面紧跟数组名时，编译器知道你要算的是整个数组占了多少内存，而不是算一个指针的大小。
- **数值：** 元素个数 * 单个元素大小。

代码示例

```
1 int arr[5] = { 1, 2, 3, 4, 5 };
2
3 // 32位系统指针占4字节，64位系统指针占8字节
4 printf("sizeof(arr) = %lu\n", sizeof(arr));
5 // 输出: 20
6 // 解释: 5个int, 每个4字节, 5 * 4 = 20。
7
8 // 对比: 如果数组名退化为指针
9 int* p = arr;
10 printf("sizeof(p) = %lu\n", sizeof(p));
11 // 输出: 8 (假设是64位系统)
12 // 解释: p 是指针变量, sizeof(p) 算的是指针变量本身的大小, 不是它指向的数据大小。
13
```

常见陷阱

如果数组名作为参数传给函数，它在函数内部会退化为指针，此时 sizeof 就救不了你了。

```
1 void func(int arr[]) { // 这里本质上变成了 int *arr
2     printf("%lu", sizeof(arr)); // 输出 8 (指针的大小, 而不是数组大小!)
3 }
```

情况二：&arr

结果： 得到“整个数组”的地址。

- **不退化：** 取地址运算符 & 作用于数组名时，取到的是整个数组的地址，而不是首元素的地址。
- **数值：** 虽然数值上 &arr 和 &arr[0] 是一样的（都是数组首字节的地址），但**类型天差地别**。

类型对比

表达式	含义	指向的类型	指针步长 (+1)
arr (通常情况)	首元素地址	int *	跳过 1 个 int (4 字节)
&arr[0]	首元素地址	int *	跳过 1 个 int (4 字节)
&arr	整个数组的地址	int (*)[5]	跳过整个数组 (20 字节)

代码验证

```
1 int arr[5] = { 1, 2, 3, 4, 5 };
2
3 printf("arr 的地址:    %p\n", (void*)arr);
4 printf("&arr 的地址:   %p\n", (void*)&arr);
5 // 输出的地址值是一样的, 比如 0x1000
6
7 printf("arr + 1:      %p\n", (void*)(arr + 1)); // 0x1004 (跳了4字节)
8 printf("&arr + 1:     %p\n", (void*)&arr + 1); // 0x1014 (跳了20字节)
9
```

解析：

- arr + 1: arr 退化为 int*, 加 1 走到下一个整数。
- &arr + 1: &arr 是指向“长度为 5 的数组”的指针，加 1 直接跳过这 5 个整数，走到这 5 个整数之后的内存位置。

总结表

表达式	数组名是否退化?	含义	结果类型	典型用途
sizeof(arr)	否	计算整个数组的大小	size_t (无符号整型)	计算数组元素个数 sizeof(arr)/sizeof(arr[0])
&arr	否	取整个数组的地址	int (*)[n] (数组指针)	处理二维数组、高级指针运算

一句话记忆

- **sizeof(arr):** 关心的是**重量**（占多少字节），所以要看整个数组。
- **&arr:** 关心的是**地盘**（整个数组的边界），所以地址等级比普通元素高一级。

sizeof (p) 的大小是几个字节取决于什么，还是说都是 8 字节

sizeof(p) 的大小取决于什么？

这是一个非常棒的问题，触及了 C 语言底层的硬件原理。

答案是：sizeof(p) 的大小取决于你使用的计算机架构（特别是操作系统和 CPU 的位数），并不是固定死的 8 字节。

但在现代主流的 64 位系统上，它确实是 8 字节。

下面我为你详细拆解决定它大小的因素。

1. 核心决定因素：地址总线的宽度

指针变量 p 的作用是存储内存地址。

内存地址其实就是一串二进制数字（比如 0x00000000 或 0xFFFFFFFF）。

- 如果 CPU 有 32 根地址线：它能表示的最大地址是 2^{32} 。存储这个地址需要 32 个 bit，也就是 4 个字节。
- 如果 CPU 有 64 根地址线：它能表示的最大地址是 2^{64} 。存储这个地址需要 64 个 bit，也就是 8 个字节。

结论： 指针的大小，本质上就是为了能装得下一个内存地址编号所需的字节数。

2. 常见情况对照表

环境 / 架构	指针大小	解释
32 位系统 (x86)	4 字节	地址范围 0~4GB。所有指针 (int*, char*, double*) 都是 4 字节。
64 位系统 (x64)	8 字节	地址范围极大。所有指针通常都是 8 字节。这是你当前最可能遇到的情况。
8 位单片机 (如 Arduino Uno)	2 字节	内存极小，地址编号很短，2 个字节就够了。
16 位系统 (DOS 时代)	2 字节	古老的标准。

3. 既然是 64 位系统，指针大小都一样吗？

虽然绝大多数情况下指针大小是固定的，但这里有两个需要区分的概念：

(1) 不同类型的指针大小一样吗？

在同一个编译环境下，所有普通指针的大小都是一样的。

不管是 int *, char *, double * 还是 struct Student *, 它们存的都是地址编号，“门牌号”长度是一样的。

```
1 printf("%zu\n", sizeof(int*));    // 输出 8
2 printf("%zu\n", sizeof(char*));   // 输出 8
3 printf("%zu\n", sizeof(double*)); // 输出 8
```

(2) 特殊指针（例外情况）

- 函数指针： 在某些特殊架构（如分段内存架构）下，函数指针的大小可能与数据指针不同，但在现代 x64 Windows/Linux 上，通常也是 8 字节。
- 成员指针 (C++)： 在 C++ 中，指向类成员的指针（如 int Class::*p）大小可能比较特殊，有时甚至是 16 字节（视编译器实现而定），但这已经属于 C++ 的进阶内容了。

4. 你的代码在什么环境下？

你现在的代码输出 8 字节，说明：

- 你使用的是 64 位的操作系统（Windows 10/11 或 macOS 或 Linux）。
 - 你的编译器正在以 64 位模式编译代码（Visual Studio 通常叫 x64，gcc 默认可能是 x64）。
- 小实验： 如果你把编译器设置改成 x86 (32 位模式)，再次运行同样的代码，你会发现输出变成了 4 字节。

总结

- sizeof(p) 不是语法规定的常量，而是硬件环境决定的。
- 32 位环境 -> 指针占 4 字节。
- 64 位环境 -> 指针占 8 字节。
- 无论指针指向什么类型的数据，在同一环境下，指针变量本身的体积是一样的。

补充一个知识点：数组名不能被赋值

在 C 语言里，数组名（比如 arr）代表的是数组在内存中的首地址。编译器把它看作一个常量。这就好比 int a = 10; 中，a 是变量，10 是常量。你不能给 10 赋值（比如写 10 = 20;），同样你也不能给数组名赋值。

错误示范：

```
1 int arr[5] = { 1, 2, 3, 4, 5 };
2 int b[5] = { 6, 7, 8, 9, 0 };
3
4 arr = b; // 错误！编译器会报错：“表达式必须是可修改的左值”
5 // arr 是数组名（常量），不能放在等号左边被修改
```

你可能会问：“那我以前写 int *p = arr; 是怎么回事？”

这叫指针赋值。你是把 arr 的地址值赋给了指针变量 p，而不是把什么东西赋给了 arr 本身。